SABET-QA: Temporal Knowledge Graph Question Answering

Brahim Touayouch[1,2,*] **Mirette Moawad**[1,*] **Dmitry Akulov**[1,*]

[1]QuickSort Research, Paris, France
[2]ENS Paris-Saclay, École Polytechnique, France

Contact: brahim.touayouch.2022@polytechnique.org

Abstract

Question Answering over Temporal Knowledge Graphs (TKGQA) requires reasoning over time-sensitive facts, yet existing embedding-based methods struggle with multi-step queries due to single-pass reasoning pipelines. We propose SABET-QA, a framework that iteratively refines reasoning states across multiple hops via a bidirectional entity-temporal scoring mechanism and a slot-aware contextualization module that aligns question semantics with temporal KG embeddings. A differentiable working memory enables progressive hypothesis refinement, while auxiliary temporal boundaries serve as coarse supervision when available. Experiments on CronQuestions, Complex-CronQuestions, MultiTQ, and TimeQuestions demonstrate consistent improvements over strong baselines, particularly on complex multi-step temporal queries.

1 Introduction

The proliferation of large-scale Knowledge Graphs (KGs) such as Wikidata (Vrandečić and Krötzsch, 2014), Freebase (Bollacker et al., 2008) and YAGO (Suchanek et al., 2007) has made Question Answering over KGs (KGQA) a crucial interface for accessing structured knowledge. However, real-world facts evolve over time, motivating Temporal Knowledge Graphs (TKGs) where a fact is represented as a quintuple $(s, r, o, [t_s, t_e])$. Temporal KGQA (TKGQA), the task of answering questions over such dynamic graphs, is essential for reasoning about a changing world.

Despite progress in static KGQA (Jiang et al., 2023; Saxena et al., 2020), TKGQA presents unique challenges. Natural language questions contain explicit temporal constraints (e.g., "Who was the president in 2008?") or implicit compositional ones (e.g., "Who was the president after Obama?"). Embedding-based approaches like CronKGQA (Saxena et al., 2021) reduce simple queries to link prediction, but struggle with complex reasoning. Later work such as TempoQR (Mavromatis et al., 2021) enriches question representations with contextualized time and entity information, yet these methods still process questions in a single forward pass, lacking iterative refinement and effective aggregation of distributed temporal evidence.

We identify three specific gaps in existing TKGQA methods:

1. **Single-Shot Reasoning:** Complex questions require sequential deduction (e.g., finding an entity, locating its tenure, then identifying the successor). Single-pass models cannot revisit or correct intermediate errors.
2. **Ambiguous Directionality:** Questions mention entities without specifying their grammatical role (head or tail) in the relation. Existing models often assume a fixed direction, leading to incorrect scoring.
3. **Context-Dependent Ambiguity:** Entities like "Washington" may refer to a person, a city, or a state depending on context. Lexical matching or fixed entity linking fails when the same mention carries different meanings.

To address these gaps, we propose **SABET-QA**, a temporal KGQA framework combining slot-aware contextualization, bidirectional entity–temporal scoring, and iterative multi-hop reasoning. When coarse temporal hints are available, SABET-QA exploits them as additional supervision. A full architectural description is provided in Section 3.3.

Our contributions are summarized as follows:

- We propose **SABET-QA**, an iterative temporal KGQA framework that progressively refines predictions through a working-memory-based multi-

[*] Work conducted at QuickSort Research, Paris, France (https://www.quicksort.fr). Correspondence to: mohamed@quicksort.fr or personal emails.

hop reasoning process.

- We introduce bidirectional entity–temporal scoring to address head–tail ambiguity and improve reasoning over questions with implicit directional structure.
- We empirically demonstrate that SABET-QA outperforms strong baselines on CronQuestions (Saxena et al., 2021), ComplexCronQuestions (Chen et al., 2022), MultiTQ (Chen et al., 2023), and TimeQuestions (Jia et al., 2021), with particularly strong gains on complex questions.

2 Related Work

Our work relates to two areas: Temporal Knowledge Graph Representations and (Temporal) Knowledge Graph Question Answering.

2.1 Temporal Knowledge Graph Representations

Embedding-based TKGQA builds on Temporal Knowledge Graph Embedding (TKGE) methods. Early approaches such as TTransE (Leblay and Chekol, 2018) extended static translation-based models (Bordes et al., 2013) by adding temporal embeddings to the scoring function. More recently, tensor decomposition methods have become prominent (Cai et al., 2023). TComplEx (Lacroix et al., 2020) extends ComplEx (Trouillon et al., 2016) to fourth-order tensors and shows that regularized decomposition can capture temporal dynamics effectively. These structured embedding spaces provide a compact latent representation of the TKG (Cai et al., 2024), enabling downstream models to reason over relational and temporal dependencies in continuous vector form rather than through explicit graph traversal.

2.2 Knowledge Graph Question Answering

Knowledge graph question answering (KGQA) has evolved along three main directions: *semantic parsing*, *neural representation learning*, and *LLM-based* approaches (Su et al., 2026).

Semantic parsing methods (Berant et al., 2013; Chen et al., 2024a; Yao and Van Durme, 2014; Bao et al., 2016) translate natural-language questions into formal logical forms or executable structured queries. They offer explicit reasoning traces and high interpretability, but depend on handcrafted grammars, schema-specific operators, or substantial annotated supervision, limiting scalability on complex question types.

Neural representation learning approaches embed questions and graph elements into a shared latent space. Early work such as KEQA (Huang et al., 2019) and EmbedKGQA (Saxena et al., 2020) treats QA as ranking over KG embeddings, avoiding explicit query construction. Later methods incorporate graph neural networks, attention mechanisms, and multi-hop reasoning to model structural dependencies and compositional questions (Sun et al., 2018; Jia et al., 2021; Liu et al., 2023; Jiao et al., 2022). These models scale better than semantic parsers and tolerate noisy or incomplete graphs, but were developed for static KGs and do not model temporal validity.

LLM-based methods extend KGQA beyond fixed templates by generating executable queries, reasoning over retrieved evidence, or combining retrieval with generation (Qian et al., 2024; Jia et al., 2024; Gao et al., 2024; Chen et al., 2024b). They reduce hand-engineered logic and improve linguistic flexibility, yet remain sensitive to retrieval quality, grounding errors, and hallucination, and are still most mature in non-temporal settings.

These limitations are amplified in temporal KGQA, where answers depend on when facts hold. The field has therefore moved from direct retrieval toward multi-step reasoning over time-sensitive constraints. CronKGQA (Saxena et al., 2021) casts a question as a *virtual relation* in a temporal embedding space, enabling link-prediction-style answering. This works for simple questions but struggles with sequential deduction or head–tail ambiguity. TempoQR (Mavromatis et al., 2021) enriches question representations with contextualized temporal and entity information, yielding stronger performance on complex questions, but its reasoning remains largely single-pass, limiting the ability to revise intermediate hypotheses. SubGTR (Chen et al., 2022) introduces subgraph-based temporal reasoning with logical constraints and highlights pseudo-temporal questions in CronQuestions. While effective when subgraph extraction is reliable, it depends heavily on extracted structures and is less robust on incomplete or noisy graphs, or when transferring across datasets.

In contrast, SABET-QA follows the embedding-based virtual-relation paradigm but adds iterative multi-hop reasoning with differentiable working memory. At each hop, the model refines a latent reasoning state, builds hop-specific relation representations, and computes intermediate scores for both entities and timestamps. To address

head–tail ambiguity, it uses bidirectional entity scoring, combining forward and backward signals through a learned gate. Unlike approaches relying on explicit subgraph extraction or dataset-specific post-processing, SABET-QA operates directly over pretrained temporal KG embeddings and natural-language questions, making it broadly applicable across benchmarks while remaining simple in design.

Recent work also explores LLM-based autonomous agents for TKGQA, but these introduce substantial inference latency, computational overhead, and dependency on non-deterministic external APIs. SABET-QA operates strictly within the dense embedding-based paradigm, providing low-latency, deterministic, and locally deployable inference. We therefore compare against embedding-based baselines as the appropriate reference class.

3 Temporal Knowledge Graph Question Answering Methods

3.1 Temporal KG Embeddings

We train the temporal knowledge graph embeddings used throughout our models with *TComplEx* (Lacroix et al., 2020). TComplEx represents entities, relations, and timestamps with complex-valued embeddings and scores a temporal fact (s, r, o, t) using a multilinear complex product:

$$\phi(s, r, o, t) = \Re\left(\langle \mathbf{u}_s, \mathbf{v}_r, \overline{\mathbf{u}}_o, \mathbf{w}_t \rangle\right),$$

where $\mathbf{u}_s$, $\mathbf{u}_o$, $\mathbf{v}_r$, and $\mathbf{w}_t$ denote the subject, object, relation, and timestamp embeddings, respectively.

The learned entity and time representationss are used to initialize the shared embedding tables across all QA models [1]. Additional implementation and training details are provided in Appendix A.

3.2 Baseline Methods

This section summarizes the main comparison baselines used in our experiments.

3.2.1 LM_TKGQA: Language Model Baseline

LM_TKGQA encodes the question with a pretrained language model such as BERT (Devlin et al., 2019) or RoBERTa (Liu et al., 2019) and projects it into the temporal knowledge graph embedding space. Candidate entities and timestamps are then ranked with a lightweight prediction head, but temporal structure is not modeled explicitly.

3.2.2 EmbedKGQA: Embedding-Based KGQA

EmbedKGQA represents the question as a soft relation and retrieves answers by matching it against knowledge graph embeddings (Saxena et al., 2020). While effective for multi-hop reasoning, it does not explicitly model temporal constraints.

3.2.3 CRONKGQA: Temporal Inference Model

CRONKGQA extends embedding-based KGQA by jointly predicting entities and timestamps using temporal knowledge graph embeddings (Saxena et al., 2021). This enables reasoning over temporal ordering and time-sensitive facts.

3.2.4 TempoQR: Contextualized Temporal Reasoning

TempoQR combines a pretrained language model, temporal knowledge graph embeddings, and a transformer-based fusion module (Mavromatis et al., 2021). This improves contextual understanding while explicitly incorporating temporal information.

3.2.5 SubGTR: Subgraph-Based Temporal Reasoning

SubGTR (Chen et al., 2022) performs reasoning over a task-relevant subgraph extracted around the query and its temporal context. Restricting inference to this neighborhood improves efficiency while exploiting local graph structure.

On postprocessing and dataset dependence. Some baselines, such as SubGTR, rely on additional postprocessing (e.g., subgraph extraction) tailored to specific datasets. In contrast, our model is designed to operate across datasets without task-specific processing, making it easier to transfer between benchmarks.

3.3 Proposed Method: SABET-QA

SABET-QA answers temporal questions through **iterative latent-state refinement** over K hops. At each hop, the model produces bidirectional entity and time scores, then updates a differentiable working memory to progressively sharpen its hypothesis. The architecture couples a frozen pretrained LM

[1] The embeddings used for the datasets in this study are provided in the supplementary material. To facilitate future work, we standardized the embedding structure across all datasets. Some embeddings were trained from scratch, whereas others were initialized from prior work.

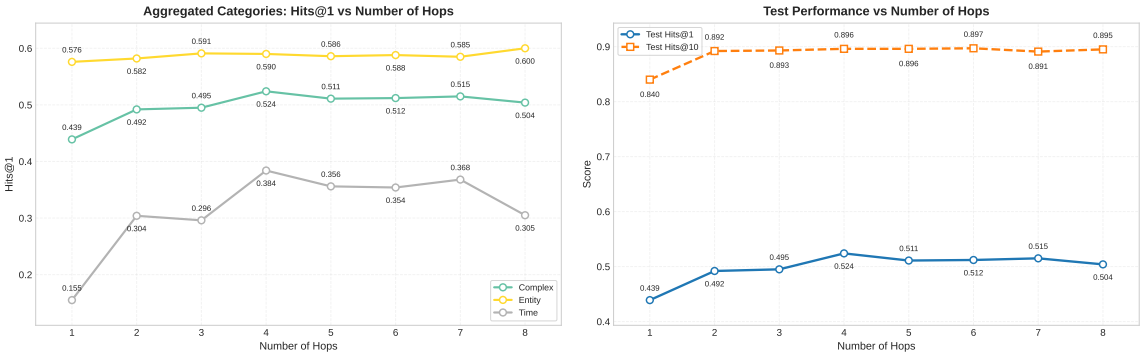


Figure 1: Impact of the number of reasoning hops on Complex-CronQuestions. Left: Hits@1 for different question categories. Right: overall test Hits@1 and Hits@10.

Model	Entity Contextualization	Bidirectional Entity & Time Scoring	Hard Supervision	Complex-CronQuestions	
				Hits@1	Hits@10
SABET-QA-Hard	✓	✓	✓	**0.807**	**0.962**
	✗	✓	✓	0.707	0.946
	✓	✗	✓	0.659	0.849
	✓	✓	✗	0.524	0.896

Table 1: Ablation study of SABET-QA-Hard on the Complex-CronQuestions test set. Removing any component degrades performance, with the largest drop observed when bidirectional entity and time scoring is disabled.

with a TComplEx temporal scorer, operating in four stages: (i) question encoding and slot contextualization, (ii) hop-specific relation projection, (iii) bidirectional scoring with memory update, and (iv) adaptive hop aggregation. Pseudocode and full dimensional details are in Appendix B.

3.3.1 Slot-Aware Question Encoding

Given tokenized question $\mathbf{x}$, a pretrained LM yields hidden states $\mathbf{H} \in \mathbb{R}^{L\times 768}$, where L is the number of tokens and 768 is the hidden representation dimension of the pretrained LM (BERT or RoBERTa). These representations are linearly projected to the TKG embedding dimension:

$$\mathbf{T} = f_{\text{text}}(\mathbf{H}) \in \mathbb{R}^{L\times D}. \tag{1}$$

where D denotes the dimensionality of the TKG embedding space (i.e., the entity and timestamp embedding dimension).

The model first identifies the entity and temporal mentions in the question. When gold annotations are available, these mentions are extracted directly; otherwise, they are obtained using a named entity recognition (NER) system. The identified entity mentions are arbitrarily assigned to the head ($\mathbf{h}$) and tail ($\mathbf{t}$) query slots, while the temporal expression is assigned to the time slot ($\boldsymbol{\tau}$). The corresponding embeddings are retrieved from the TKBC embedding tables, with learned dummy embeddings used for any missing slots. These query embeddings are then contextualized via multi-head cross-attention (Vaswani et al., 2023) over the projected question token representations, followed by a residual gating mechanism:

$$[\mathbf{h}', \mathbf{t}', \boldsymbol{\tau}'] = \text{Gate}(\text{MHA}([\mathbf{h}, \mathbf{t}, \boldsymbol{\tau}], \mathbf{T}, \mathbf{T})) . \tag{2}$$

A global question representation $\mathbf{s}$ is constructed by concatenating the hidden representation of the special classification token ([CLS]), the mean-pooled token representations, and the max-pooled token representations of $\mathbf{T}$. The resulting vector is projected to obtain the base reasoning state:

$$\mathbf{z}^{(0)} = f_{\text{rel}}\big([\mathbf{s}; \mathbf{h}'; \mathbf{t}'; \boldsymbol{\tau}']\big). \tag{3}$$

3.3.2 Iterative Hop-Wise Reasoning with Bidirectional Scoring

At hop k, the current latent state $\mathbf{z}^{(k-1)}$ and summary $\mathbf{s}$ are fused into a hop-specific relation vector $\mathbf{r}^{(k)} = f_{\text{hop}}^{(k)}([\mathbf{z}^{(k-1)}; \mathbf{s}])$, then split into entity-oriented and time-oriented projections: $\mathbf{r}_{\text{ent}}^{(k)}$ and $\mathbf{r}_{\text{time}}^{(k)}$.

Model	Frozen LM	Frozen TKE	Complex-CronQuestions		CronQuestions	
			Hits@1	Hits@10	Hits@1	Hits@10
SABET-QA	✓	✓	0.524	**0.896**	**0.843**	**0.969**
	✓	✗	0.446	0.871	0.773	0.950
	✗	✓	**0.547**	0.892	0.836	0.968
	✗	✗	0.476	0.882	0.784	0.946
SABET-QA-Hard	✓	✓	**0.807**	0.962	**0.954**	**0.989**
	✓	✗	0.759	0.958	0.925	0.983
	✗	✓	0.803	**0.964**	0.949	0.988
	✗	✗	0.769	0.959	0.902	0.974

Table 2: Ablation study evaluating the impact of freezing the language model (LM) and temporal knowledge graph embedding (TKE) modules under both no supervision (**SABET-QA**) and hard supervision (**SABET-QA-Hard**). The comparison isolates the contribution of each component and demonstrates the effect of unfreezing the parameters on overall reasoning performance.

Temporal hint injection (optional). When auxiliary temporal boundaries (t_1, t_2) are available, we provide them as hard supervision (denoted by the *hard* suffix in the model name). Following TempoQR (Mavromatis et al., 2021), these boundaries are derived from the earliest and latest timestamps of facts involving the question entities and are injected into the working memory through a learned gating mechanism:

$$\mathbf{z}^{(k-1)} \leftarrow \mathbf{z}^{(k-1)} + \boldsymbol{\gamma}_{\text{time}}^{(k)} \odot (\mathbf{e}_{t_1} + \mathbf{e}_{t_2}). \quad (4)$$

Bidirectional scoring. To resolve head-tail ambiguity, entity candidates are scored in both directions using the contextualized slots, then fused by a summary-conditioned gate $\alpha_{\text{ent}}^{(k)} = \sigma(W_{\text{ent}}\mathbf{s})$:

$$\mathbf{s}_{\text{ent}}^{(k)} = \alpha_{\text{ent}}^{(k)} \cdot \text{Score}_{\text{TComplEx}}(\mathbf{h}', \mathbf{t}', \mathbf{r}_{\text{ent}}^{(k)}, \boldsymbol{\tau}') + (1 - \alpha_{\text{ent}}^{(k)}) \cdot \text{Score}_{\text{TComplEx}}(\mathbf{t}', \mathbf{h}', \mathbf{r}_{\text{ent}}^{(k)}, \boldsymbol{\tau}') \quad (5)$$

Timestamp candidates are scored analogously (without the time slot in the scorer) and fused via $\alpha_{\text{time}}^{(k)}$.

3.3.3 Working-Memory Update and Aggregation

After scoring, soft distributions $\mathbf{p}_{\text{ent}}^{(k)} = \text{softmax}(\mathbf{s}_{\text{ent}}^{(k)})$ and $\mathbf{p}_{\text{time}}^{(k)} = \text{softmax}(\mathbf{s}_{\text{time}}^{(k)})$ yield expected embeddings:

$$\bar{\mathbf{e}}_{\text{ent}}^{(k)} = \mathbf{p}_{\text{ent}}^{(k)\top}\mathbf{E}_{\text{ent}}, \quad \bar{\mathbf{e}}_{\text{time}}^{(k)} = \mathbf{p}_{\text{time}}^{(k)\top}\mathbf{E}_{\text{time}}. \quad (6)$$

Their sum is projected to a memory vector $\mathbf{m}^{(k)}$ that refines the latent state:

$$\mathbf{z}^{(k)} = \text{Gate}\big(\text{Attn}(\mathbf{z}^{(k-1)}, \mathbf{m}^{(k)})\big). \quad (7)$$

This lets the model carry forward soft predictions from earlier hops, progressively refining its hypothesis.

Adaptive aggregation. A hop selector $\boldsymbol{\beta} = \text{softmax}(W_{\text{hop}}\mathbf{s})$ weights the K hop-specific score vectors:

$$\mathbf{s}_{\text{ent}} = \sum_{k=1}^{K} \beta_k \mathbf{s}_{\text{ent}}^{(k)}, \qquad \mathbf{s}_{\text{time}} = \sum_{k=1}^{K} \beta_k \mathbf{s}_{\text{time}}^{(k)}. \quad (8)$$

The concatenated output $[\mathbf{s}_{\text{ent}}; \mathbf{s}_{\text{time}}]$ is trained with cross-entropy against the gold answer. TKBC embeddings remain fixed during QA training unless the unfrozen ablation setting is enabled.

4 Experiments

4.1 Datasets

We evaluate SABET-QA on four benchmark datasets for temporal question answering: CronQuestions (Saxena et al., 2021), Complex-CronQuestions (Chen et al., 2022), MultiTQ (Chen et al., 2023), and TimeQuestions (Jia et al., 2021). Together, these benchmarks cover a broad spectrum of temporal reasoning settings, including temporal entity prediction, timestamp prediction, temporal comparison, and multi-hop reasoning over temporal knowledge graphs.

CronQuestions and Complex-CronQuestions are synthetic benchmarks derived from temporal Wikidata facts and are designed to assess compositional temporal reasoning. MultiTQ is a large-scale automatically generated dataset containing questions that involve diverse temporal operators and more intricate reasoning patterns. TimeQuestions comprises natural-language temporal questions collected from real-world sources, offering a complementary evaluation setting that more closely reflects realistic user queries.

Detailed dataset statistics, temporal knowledge graph characteristics, and answer-type distributions are provided in Appendix C (Tables 6, 7, and 8).

4.2 Method Evaluation

We evaluate SABET-QA using the standard ranking-based metrics commonly adopted in the temporal question answering literature, namely *Hits@1* and *Hits@10*. Hits@1 measures the proportion of queries for which the correct answer is ranked first, whereas Hits@10 measures the proportion of queries for which the correct answer appears within the top ten ranked candidates. Following the evaluation protocols of the respective benchmark datasets, we report overall performance as well as disaggregated results by question complexity (when available) and answer type (entity versus timestamp).

Table 3 reports results on CronQuestions. SABET-QA achieves **0.843** Hits@1, improving over TempoQR by **4.7** points, a gain concentrated almost entirely on the complex subset (**+7.5** over TempoQR's 0.658). This pattern, strong gains on compositional reasoning with maintained or improved performance on simple queries, recurs across all four benchmarks and validates our core hypothesis: iterative refinement with bidirectional scoring is particularly effective when questions require sequential temporal deduction.

The *hard-supervised* variant (SABET-QA-Hard) pushes performance to **0.954** Hits@1 on CronQuestions and **0.807** on Complex-CronQuestions (Table 4). The margin over TempoQR-Hard widens to **17.5** points on Complex-CronQuestions, suggesting that our architecture better exploits coarse temporal boundaries when reasoning chains are longer. Notably, SABET-QA-Hard's time-answer accuracy on Complex-CronQuestions (**0.931** Hits@1) approaches its entity-answer accuracy (**0.747**), whereas TempoQR-Hard exhibits a **27.3**-point gap between the two. We attribute this to the working-memory mechanism, which propagates intermediate temporal hypotheses across hops rather than scoring time and entity candidates independently.

On MultiTQ (Table 5), absolute scores are lower across all methods, reflecting the dataset's coarser temporal granularity and more diverse operator vocabulary. SABET-QA still leads all baselines, with the largest relative improvement on timestamp prediction (**0.111** vs. Bert's **0.069** Hits@1).

TimeQuestions (Table 5) presents naturalistic questions with noisy entity linking. Here SABET-QA outperforms TempoQR by **9.3** Hits@1 points overall, with the non-hard variant actually edging SABET-QA-Hard on time-specific accuracy (**0.609** vs. **0.604**).

Overall, SABET-QA establishes the best results on all four benchmarks. The gains are largest on complex multi-hop questions and timestamp prediction, confirming that iterative, structure-aware temporal reasoning outperforms single-pass embedding matching.

4.3 Method Behaviour Analysis

Varying the number of hops. Figure 1 (left) plots per-category Hits@1 on Complex-CronQuestions as K increases from 1 to 8. Three distinct regimes emerge. *Complex* queries rise sharply from $K{=}1$ (**0.439**) to $K{=}4$ (**0.524**), then fluctuate in a narrow band (**0.504** - **0.515**) through $K{=}8$; the peak at $K{=}4$ represents a **19.4**% relative gain over single-hop reasoning. *Time* queries follow a similar trajectory, peaking at $K{=}4$ (**0.384**) before degrading to **0.305** at $K{=}8$.

Entity queries behave differently: they rise monotonically from $K{=}1$ (**0.576**) to $K{=}8$ (**0.600**), with only marginal gains beyond $K{=}4$. This divergence is structurally informative: complex and time queries benefit from *moderate* iterative refinement enough to resolve intermediate ambiguities, but not so deep that compounding complexity dominates whereas entity queries, which require less compositional reasoning, tolerate and even profit from extended unrolling.

The right panel confirms that overall Hits@1 mirrors the complex category, peaking at $K{=}4$ (**0.524**). Hits@10 behaves differently: it rises steeply from $K{=}1$ (**0.840**) to $K{=}2$ (**0.892**), then plateaus from $K{=}4$ onward (**0.895 — 0.897**). This decoupling Hits@1 saturating earlier than Hits@10 indicates that additional hops primarily improve ranking quality within the top 10 rather than top-1 precision. We use $K{=}4$ in all reported results as the best compromise between accuracy and computational cost.

Hop Attention Analysis and Iterative Refinement. To better understand how SABET-QA exploits its multi-hop reasoning mechanism, we analyze the aggregation weights (β_k) and top-1 hop selections over $30,000$ validation queries from the CronQuestions dataset. As shown in Figure 2(a,b), the model consistently favors deeper reasoning states: the mean aggregation weight increases

Model	Hits@1					Hits@10				
	Overall	Complex	Simple	Entity	Time	Overall	Complex	Simple	Entity	Time
BERT	0.257	0.253	0.262	0.292	0.191	0.642	0.617	0.676	0.650	0.627
CronKGQA	0.646	0.391	0.987	0.698	0.550	0.886	0.806	0.993	0.901	0.858
EmbedKGQA	0.433	0.370	0.516	0.576	0.166	0.787	0.735	0.857	0.892	0.592
TempoQR	0.796	0.658	0.981	0.880	0.640	0.959	0.934	0.992	0.975	0.930
TempoQR-Hard	0.914	0.861	0.984	0.923	0.896	0.979	0.968	0.993	0.982	0.973
SubGTR-Hard	0.913	0.859	0.984	0.917	0.904	0.980	0.970	0.993	0.982	0.975
SABET-QA	**0.843**	**0.733**	**0.989**	**0.882**	**0.770**	**0.969**	**0.953**	**0.994**	**0.979**	**0.954**
SABET-QA-Hard	**0.954**	**0.926**	**0.994**	**0.941**	**0.980**	**0.989**	**0.983**	**0.996**	**0.986**	**0.994**

Table 3: Comparison against other methods on the CronQuestions test set. Metrics are reported for overall performance, question type (complex/simple), and answer type (entity/time).

Model	Hits@1			Hits@10		
	Overall	Entity	Time	Overall	Entity	Time
BERT	0.087	0.097	0.068	0.421	0.351	0.567
CronKGQA	0.288	0.365	0.129	0.736	0.758	0.689
EmbedKGQA	0.260	0.361	0.050	0.618	0.742	0.360
TempoQR	0.438	0.585	0.132	0.853	0.906	0.743
TempoQR-Hard	0.632	0.721	0.448	0.933	0.942	0.914
SubGTR-Hard	0.623	0.719	0.422	0.928	0.944	0.895
SABET-QA	0.524	0.590	0.384	0.896	0.925	0.836
SABET-QA-Hard	**0.807**	**0.747**	**0.931**	**0.962**	**0.955**	**0.978**

Table 4: Comparison on the Complex-CronQuestions test set. Metrics are reported for overall performance and answer type (entity/time).

monotonically from 0.137 (Hop 0) to 0.400 (Hop 3), while Hop 3 is selected as the dominant reasoning state in 56.5% of the queries.

Further stratification by question type (Figure 2(c)) reveals adaptive reasoning depth across different temporal reasoning tasks. Whereas relatively simple query types distribute attention more evenly across hops, compositionally complex queries such as *first_last* (81.6% Hop 3 selection) and *before_after* (54.3% Hop 3 selection) place the majority of their attention on the final hop. These results indicate that the differentiable working memory learns to postpone final predictions until sufficient multi-step temporal evidence has been accumulated, demonstrating its ability to adapt computation depth according to reasoning complexity.

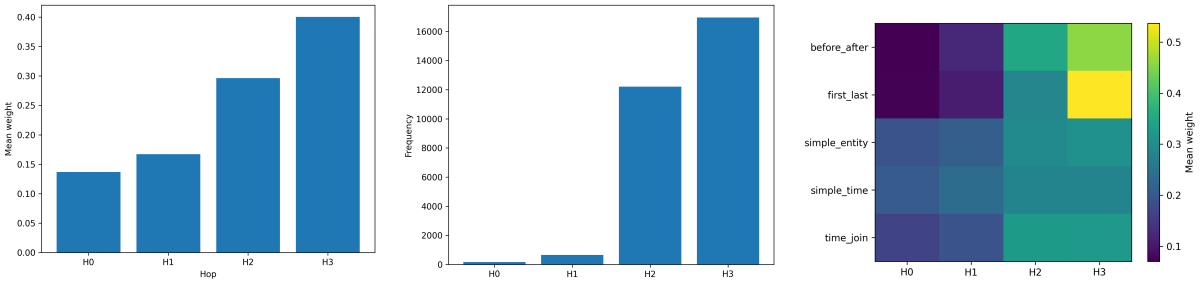
Ablation of architectural components. Table 1 isolates the contribution of each design choice in SABET-QA-Hard on Complex-CronQuestions. Removing entity contextualization (EC) drops Hits@1 by **10.0** points (**0.807** $\rightarrow$ **0.707**), showing that grounding slot representations in question text is essential when relations are lexically ambiguous. Removing bidirectional entity–time scoring (BETS) causes the largest single-component drop (**–14.8** points, to **0.659**), confirming that forward and backward scorers capture complementary directional biases; neither direction alone suffices for head–tail disambiguation. Removing hard supervision (HS) degrades performance to the non-hard variant level (**0.524**), demonstrating that temporal hint injection provides orthogonal signal when interval boundaries are available. Crucially, no partial configuration approaches the full model; the gain over the best ablated variant is **18.3** Hits@1 points, establishing that slot contextualization, bidirectional scoring, and hard supervision are *jointly* necessary rather than redundant.

Which pretrained modules should be fine-tuned? Table 2 examines whether updating the pretrained language model (LM) and temporal knowledge embeddings (TKE) during QA training improves performance.

Under *no hard supervision* (SABET-QA, top block), the best Complex-CronQuestions result (**0.547** Hits@1) is obtained by *unfreezing the LM while keeping TKE frozen* outperforming the fully frozen baseline (**0.524**) by **2.3** points. Unfreezing TKE alone (**0.446**) or both modules (**0.476**) underperforms the frozen baseline, suggesting that updating TKE without hard boundaries introduces

Model	MultiTQ						TimeQuestions					
	Hits@1			Hits@10			Hits@1			Hits@10		
	Overall	Entity	Time	Overall	Entity	Time	Overall	Entity	Time	Overall	Entity	Time
BERT	0.103	0.117	0.069	0.516	0.632	0.234	0.450	0.409	0.556	0.569	0.520	0.698
CronKGQA	0.278	0.387	0.011	0.527	0.724	0.046	0.326	0.296	0.405	0.454	0.409	0.569
EmbedKGQA	0.243	0.342	0.002	0.489	0.685	0.012	0.288	0.266	0.344	0.468	0.426	0.577
TempoQR	0.327	0.456	0.014	0.571	0.783	0.055	0.409	0.406	0.416	0.530	0.513	0.574
TempoQR-Hard	0.335	0.465	0.018	0.579	0.788	0.068	0.410	0.411	0.407	0.528	0.511	0.572
SubGTR-Hard	0.337	0.469	0.015	0.576	0.789	0.056	0.419	0.415	0.427	0.532	0.512	0.583
SABET-QA	0.373	**0.480**	0.111	0.700	0.804	0.444	0.502	0.500	**0.609**	**0.619**	**0.568**	**0.750**
SABET-QA-Hard	**0.403**	0.479	**0.219**	**0.715**	**0.810**	**0.485**	**0.504**	**0.513**	0.604	0.615	0.565	0.747

Table 5: Comparison on the MultiTQ and TimeQuestions test sets.



(a) Mean aggregation weight (β_k). (b) Top-1 hop selection count. (c) Hop weight heatmap by query type.

Figure 2: Behavioral dynamics of the differentiable working memory ($K = 4$) across $30,000$ CronQuestions validation instances. The network dynamically routes computation depth, shifting attention mass to H3 for compositional temporal queries.

noise into an already well-structured embedding space.

Under *hard supervision* (SABET-QA-Hard, bottom block), the fully frozen configuration already achieves **0.807** Hits@1. Unfreezing the LM alone yields **0.803** (**–0.4**), a difference that is likely not significant, while unfreezing TKE alone (**0.759**) or both (**0.769**) degrades performance. The same ordering holds on CronQuestions: frozen $\geqslant$ LM-unfrozen $>$ both-unfrozen $>$ TKE-unfrozen.

These results indicate that the pretrained TComplEx embeddings are already well-optimized for temporal link prediction, and updating them during QA fine-tuning hurts generalization. The bottleneck is instead the *alignment* between the LM's question encoding and the frozen TKG scoring space. With hard supervision, this alignment is already strong enough that unfreezing the LM provides no benefit; under weak supervision, modest gains from an unfrozen LM are possible, but they are quickly overshadowed once hard temporal boundaries are available.

5 Conclusion

In this work, we presented SABET-QA, an iterative multi-hop framework for Temporal Knowledge Graph Question Answering. By introducing bidirectional entity and time scoring, our model mitigates head-tail directional ambiguity, a common failure mode in prior methods. A differentiable working memory progressively refines the latent reasoning state across hops, moving beyond single-pass inference, while slot-aware contextualization keeps entity and temporal representations grounded in question semantics throughout.

Experiments on CronQuestions, Complex-CronQuestions, MultiTQ, and TimeQuestions show consistent improvements over strong baselines, with the largest gains on complex questions. This validates our hypothesis that iterative refinement and bidirectional temporal scoring are critical for multi-step temporal reasoning.

Limitations

Despite these results, SABET-QA has several limitations. First, the approach relies on pretrained temporal KG embeddings and frozen language-model/KG components during QA training, which may constrain adaptability to new domains or evolving graph structures. Third, the method assumes access to entity and timestamp grounding when available, so performance drops when such annotations are missing or unreliable. When gold entity mentions are unavailable, SABET-QA relies on downstream NER systems (e.g., Flair (Akbik et al., 2019) on MultiTQ). Error propagation from noisy slot extractions inherently introduces a performance degradation compared to gold-annotated mentions, highlighting a dependency on upstream entity linking accuracy in end-to-end setups. Finally, the iterative multi-hop design adds architectural complexity and may increase computational cost compared with simpler single-pass models. These points are consistent with the paper's training setup and the way the model is grounded in pretrained TComplEx-based representations.

Ethical Considerations

This work is based on structured benchmark data and is intended for research on temporal reasoning. It is also important to note that the model may inherit biases, gaps, or factual errors from the underlying knowledge graphs and pretrained embeddings, and such issues can affect prediction quality. In addition, because the model is evaluated on benchmark datasets rather than sensitive personal data, the main ethical concerns relate to reproducibility, disclosure, and responsible interpretation of results rather than privacy.

References

Alan Akbik, Tanja Bergmann, Duncan Blythe, Kashif Rasul, Stefan Schweter, and Roland Vollgraf. 2019. FLAIR: An easy-to-use framework for state-of-the-art NLP. In *NAACL 2019, 2019 Annual Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, pages 54–59.

Junwei Bao, Nan Duan, Zhao Yan, Ming Zhou, and Tiejun Zhao. 2016. Constraint-based question answering with knowledge graph. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 2503–2514, Osaka, Japan. The COLING 2016 Organizing Committee.

Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. 2013. Semantic parsing on Freebase from question-answer pairs. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1533–1544, Seattle, Washington, USA. Association for Computational Linguistics.

Kurt Bollacker, Colin Evans, Praveen Paritosh, Tim Sturge, and Jamie Taylor. 2008. Freebase: a collaboratively created graph database for structuring human knowledge. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, SIGMOD '08, page 1247–1250, New York, NY, USA. Association for Computing Machinery.

Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc.

Borui Cai, Yong Xiang, Longxiang Gao, He Zhang, Yunfeng Li, and Jianxin Li. 2023. Temporal knowledge graph completion: A survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, IJCAI-2023, page 6545–6553. International Joint Conferences on Artificial Intelligence Organization.

Li Cai, Xin Mao, Yuhao Zhou, Zhaoguang Long, Changxu Wu, and Man Lan. 2024. A survey on temporal knowledge graph: Representation learning and applications. *Preprint*, arXiv:2403.04782.

Zhuo Chen, Zhao Zhang, Zixuan Li, Fei Wang, Yutao Zeng, Xiaolong Jin, and Yongjun Xu. 2024a. Self-improvement programming for temporal knowledge graph question answering. *Preprint*, arXiv:2404.01720.

Ziyang Chen, Dongfang Li, Xiang Zhao, Baotian Hu, and Min Zhang. 2024b. Temporal knowledge question answering via abstract reasoning induction. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4872–4889, Bangkok, Thailand. Association for Computational Linguistics.

Ziyang Chen, Jinzhi Liao, and Xiang Zhao. 2023. Multi-granularity temporal question answering over knowledge graphs. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11378–11392, Toronto, Canada. Association for Computational Linguistics.

Ziyang Chen, Xiang Zhao, Jinzhi Liao, Xinyi Li, and Evangelos Kanoulas. 2022. Temporal knowledge graph question answering via subgraph reasoning. *Knowledge-Based Systems*, 251:109134.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. *Preprint*, arXiv:1810.04805.

Yifu Gao, Linbo Qiao, Zhigang Kan, Zhihua Wen, Yongquan He, and Dongsheng Li. 2024. Two-stage generative question answering on temporal knowledge graph using large language models. *Preprint*, arXiv:2402.16568.

Xiao Huang, Jingyuan Zhang, Dingcheng Li, and Ping Li. 2019. Knowledge graph embedding based question answering. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*, WSDM '19, page 105–113, New York, NY, USA. Association for Computing Machinery.

Prachi Jain, Sushant Rathi, Mausam, and Soumen Chakrabarti. 2020. Temporal Knowledge Base Completion: New Algorithms and Evaluation Protocols. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3733–3747, Online. Association for Computational Linguistics.

Zhen Jia, Philipp Christmann, and Gerhard Weikum. 2024. Faithful temporal question answering over heterogeneous sources. *Preprint*, arXiv:2402.15400.

Zhen Jia, Soumajit Pramanik, Rishiraj Saha Roy, and Gerhard Weikum. 2021. Complex temporal question answering on knowledge graphs. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, CIKM '21, page 792–802. ACM.

Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, and Ji-Rong Wen. 2023. Unikgqa: Unified retrieval and reasoning for solving multi-hop question answering over knowledge graph. *Preprint*, arXiv:2212.00959.

Songlin Jiao, Zhenfang Zhu, Wenqing Wu, Zicheng Zuo, Jiangtao Qi, Wenling Wang, Guangyuan Zhang, and Peiyu Liu. 2022. An improving reasoning network for complex question answering over temporal knowledge graphs. *Applied Intelligence*, 53(7):8195–8208.

Diederik P. Kingma and Jimmy Ba. 2017. Adam: A method for stochastic optimization. *Preprint*, arXiv:1412.6980.

Timothée Lacroix, Guillaume Obozinski, and Nicolas Usunier. 2020. Tensor decompositions for temporal knowledge base completion. *Preprint*, arXiv:2004.04926.

Julien Leblay and Melisachew Wudage Chekol. 2018. Deriving validity time in knowledge graph. *Companion Proceedings of the The Web Conference 2018*.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *Preprint*, arXiv:1907.11692.

Yonghao Liu, Di Liang, Mengyu Li, Fausto Giunchiglia, Ximing Li, Sirui Wang, Wei Wu, Lan Huang, Xiaoyue Feng, and Renchu Guan. 2023. Local and global: Temporal question answering via information fusion. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23*, pages 5141–5149. International Joint Conferences on Artificial Intelligence Organization. Main Track.

Costas Mavromatis, Prasanna Lakkur Subramanyam, Vassilis N. Ioannidis, Soji Adeshina, Phillip R. Howard, Tetiana Grinberg, Nagib Hakim, and George Karypis. 2021. Tempoqr: Temporal question reasoning over knowledge graphs. *Preprint*, arXiv:2112.05785.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, and 2 others. 2019. Pytorch: An imperative style, high-performance deep learning library. *Preprint*, arXiv:1912.01703.

Xinying Qian, Ying Zhang, Yu Zhao, Baohang Zhou, Xuhui Sui, Li Zhang, and Kehui Song. 2024. TimeR^4 : Time-aware retrieval-augmented large language models for temporal knowledge graph question answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 6942–6952, Miami, Florida, USA. Association for Computational Linguistics.

Daniel Ruffinelli, Samuel Broscheit, and Rainer Gemulla. 2020. You can teach an old dog new tricks! on training knowledge graph embeddings. In *International Conference on Learning Representations*.

Apoorv Saxena, Soumen Chakrabarti, and Partha Talukdar. 2021. Question answering over temporal knowledge graphs. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6663–6676, Online. Association for Computational Linguistics.

Apoorv Saxena, Aditay Tripathi, and Partha Talukdar. 2020. Improving multi-hop question answering over knowledge graphs using knowledge base embeddings. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4498–4507, Online. Association for Computational Linguistics.

Miao Su, Zixuan Li, Zhuo Chen, Long Bai, Xiaolong Jin, and Jiafeng Guo. 2026. Temporal knowledge graph question answering: A survey. *Preprint*, arXiv:2406.14191.

Fabian M. Suchanek, Gjergji Kasneci, and Gerhard Weikum. 2007. Yago: a core of semantic knowledge. In *Proceedings of the 16th International Conference on World Wide Web*, WWW '07, page 697–706, New York, NY, USA. Association for Computing Machinery.

Haitian Sun, Bhuwan Dhingra, Manzil Zaheer, Kathryn Mazaitis, Ruslan Salakhutdinov, and William Cohen. 2018. Open domain question answering using early fusion of knowledge bases and text. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 4231–4242, Brussels, Belgium. Association for Computational Linguistics.

Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. 2016. Complex embeddings for simple link prediction. *Preprint*, arXiv:1606.06357.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention is all you need. *Preprint*, arXiv:1706.03762.

Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: a free collaborative knowledgebase. *Commun. ACM*, 57(10):78–85.

Xuchen Yao and Benjamin Van Durme. 2014. Information extraction over structured data: Question answering with Freebase. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 956–966, Baltimore, Maryland. Association for Computational Linguistics.

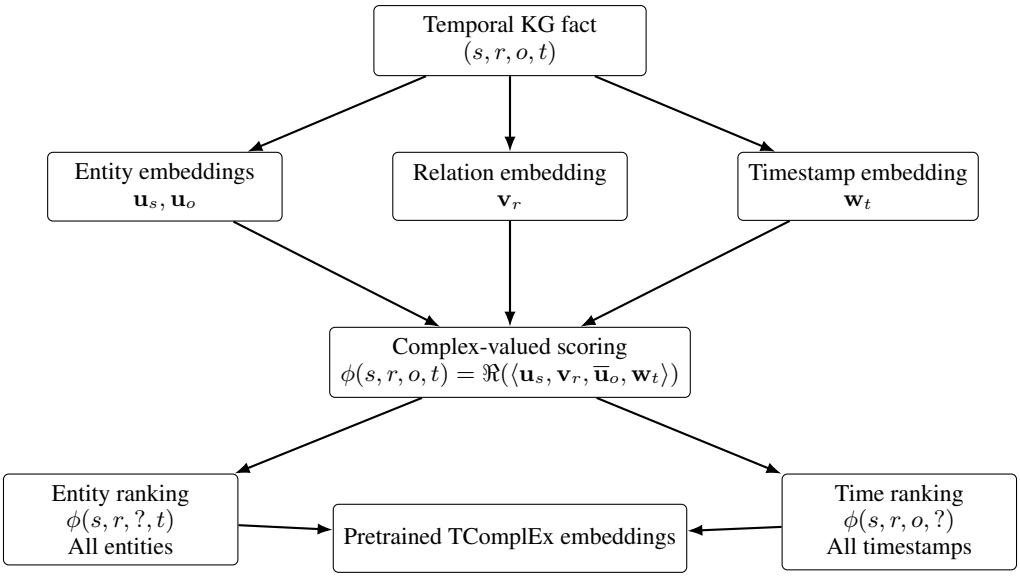


Figure 3: TComplEx pipeline used to initialize the temporal KG embeddings. The model learns complex-valued embeddings for entities, relations, and timestamps, and supports both entity ranking and timestamp ranking through the same compositional scoring function.

A Temporal KG Embedding Model

To train the temporal knowledge-graph embeddings used in our QA models, we adopt *TComplEx* (Lacroix et al., 2020), following a broader family of complex-valued embedding models for knowledge graphs. In particular, ComplEx (Trouillon et al., 2016) represents entities and relations as complex vectors and scores a fact via the real part of a multilinear product, while TComplEx extends this formulation by introducing a timestamp embedding for temporal facts. Related temporal variants such as TNTComplEx and TimePlex further enrich this framework with additional time-aware parameterizations of relations and temporal representations (Lacroix et al., 2020; Jain et al., 2020).

We use TComplEx as the default KG embedding backend for all QA models in order to ensure a controlled comparison across methods, since our focus is on evaluating the QA architecture rather than the choice of temporal KGE model. This design choice is therefore not essential to the proposed QA framework and can be replaced by another temporal embedding method without changing the core model. Nevertheless, TComplEx is a natural and widely used choice in the literature, which makes it suitable for our experiments (Ruffinelli et al., 2020).

Figure 3 summarizes the TComplEx pipeline. The model maps a temporal fact to entity, relation, and timestamp embeddings, combines them through complex-valued interactions, and supports both entity prediction and timestamp prediction.

For the MultiTQ dataset, temporal answers can be expressed at different granularities, including specific days, months, or years. To accommodate this variability, we trained the temporal knowledge graph embeddings on an augmented version of the original TKG. Specifically, for each temporal fact, we added additional facts corresponding to coarser temporal resolutions by converting timestamps into their associated month- and year-level representations. For example, a fact associated with a specific day was duplicated with equivalent month-level and year-level timestamps.

B SABET-QA Details

This appendix provides the complete mathematical specification and pseudocode for SABET-QA, matching the implementation provided in the anonymized supplementary material.

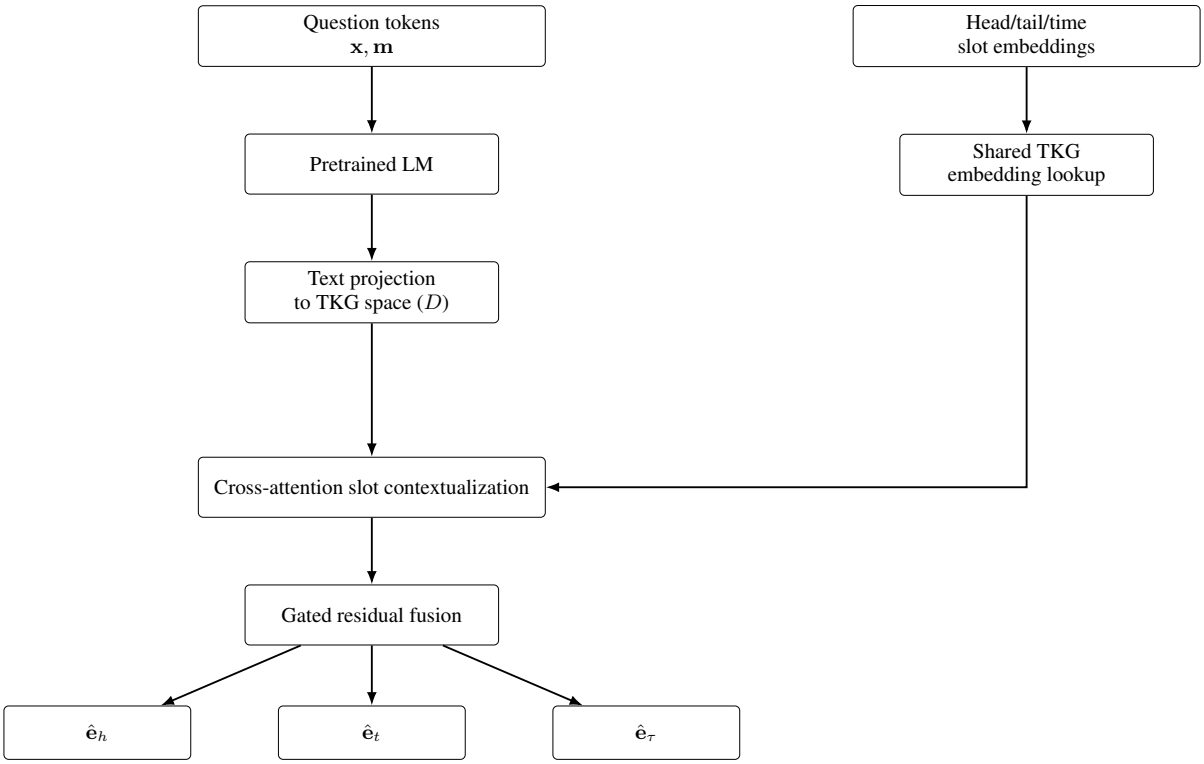


Figure 4: Question encoding and slot contextualization in SABER-TQA. Question tokens are encoded by a pretrained language model and projected into the temporal KG embedding space. Head, tail, and temporal slot embeddings are retrieved from the shared TKG table and attend to the question representation, followed by gated residual fusion to produce contextualized role-specific vectors.

B.1 Question Encoding and Slot Contextualization

Given tokenized question $\mathbf{x} = (x_1, \ldots, x_L)$ and attention mask $\mathbf{m}$, the pretrained LM produces hidden states $\mathbf{H} = \mathrm{LM}(\mathbf{x}, \mathbf{m}) \in \mathbb{R}^{L \times 768}$. These are projected into the TKG space by a feed-forward network f_{text} (linear $\rightarrow$ LayerNorm $\rightarrow$ GELU $\rightarrow$ dropout):

$$\mathbf{T} = f_{\text{text}}(\mathbf{H}) \in \mathbb{R}^{L \times D}. \tag{9}$$

Head, tail, and timestamp slot embeddings $\mathbf{e}_h, \mathbf{e}_t, \mathbf{e}_\tau$ are retrieved from the shared entity/time embedding table. They are stacked as queries for multi-head cross-attention over the projected question tokens:

$$\mathbf{C} = \mathrm{Attn}\big([\mathbf{e}_h; \mathbf{e}_t; \mathbf{e}_\tau], \mathbf{T}, \mathbf{T}\big), \tag{10}$$

with key padding derived from $\mathbf{m}$. The outputs $\mathbf{c}_h, \mathbf{c}_t, \mathbf{c}_\tau$ are layer-normalized and fused with gated residuals:

$$g_h = \sigma\big(W_g[\mathbf{e}_h; \mathbf{c}_h]\big), \qquad \hat{\mathbf{e}}_h = g_h \odot \mathbf{e}_h + (1 - g_h) \odot \mathbf{c}_h, \tag{11}$$

and analogously for $\hat{\mathbf{e}}_t$ and $\hat{\mathbf{e}}_\tau$.

B.2 Global Summary and Base Relation

A global summary vector is built from the projected token sequence using the concatenation of the [CLS] representation, masked mean pooling, and masked max pooling:

$$\mathbf{s} = f_{\text{sum}}\big([\mathbf{T}_{\text{CLS}}; \mathrm{Mean}(\mathbf{T}); \mathrm{Max}(\mathbf{T})]\big) \in \mathbb{R}^D, \tag{12}$$

where f_{sum} is a learned projection. The base latent state is then constructed from the summary and the contextualized slots:

$$\mathbf{z}^{(0)} = f_{\text{rel}}\big([\mathbf{s}; \hat{\mathbf{e}}_h; \hat{\mathbf{e}}_t; \hat{\mathbf{e}}_\tau]\big) \in \mathbb{R}^D, \tag{13}$$

where f_{rel} is a two-layer MLP ($4D \rightarrow 2D \rightarrow D$) with GELU and dropout.

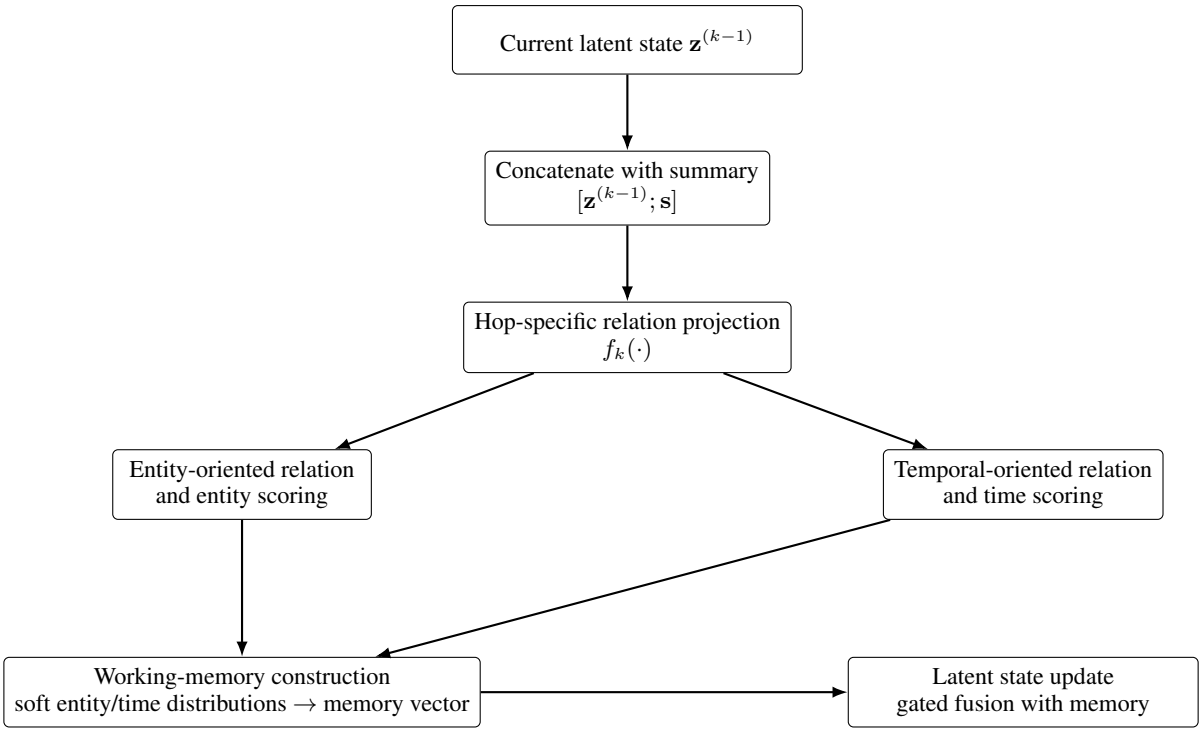


Figure 5: Iterative hop-wise reasoning in SABET-QA. At each hop, the current latent reasoning state is combined with the global question summary and transformed into a hop-specific relation representation. Dedicated entity-oriented and temporal-oriented projections generate entity and timestamp scores, whose soft distributions are converted into a working-memory representation used to update the latent state for the next reasoning hop.

B.3 Hop-Specific Relation Projection

At hop k, the current state and the global summary are concatenated and projected by a hop-specific MLP f_k:

$$\mathbf{r}^{(k)} = f_k\big([\mathbf{z}^{(k-1)}; \mathbf{s}]\big), \tag{14}$$

where f_k maps $2D \to 2D \to D$ with LayerNorm, GELU, and dropout.

This relation is then factorized into an entity-oriented view and a time-oriented view using dedicated projection heads:

$$\mathbf{r}^{(k)}_{\text{ent}} = g_{\text{ent}}(\mathbf{r}^{(k)}), \qquad \mathbf{r}^{(k)}_{\text{time}} = g_{\text{time}}(\mathbf{r}^{(k)}), \tag{15}$$

with each head implemented as a small MLP over D-dimensional inputs.

B.4 Optional Temporal Hint Injection

When auxiliary temporal endpoints (t_1, t_2) are available, their TKG embeddings are used as a hard temporal hint. Let $\mathbf{e}_{t_1}$ and $\mathbf{e}_{t_2}$ denote the corresponding timestamp embeddings. A learned gate controls how much of this hint is injected into the current latent state:

$$\boldsymbol{\tau}^{(k)} = \sigma\big(W_\tau[\mathbf{z}^{(k-1)}; \mathbf{e}_{t_1}; \mathbf{e}_{t_2}]\big), \tag{16}$$

and the state is updated as

$$\mathbf{z}^{(k-1)} \leftarrow \mathbf{z}^{(k-1)} + \boldsymbol{\tau}^{(k)} \odot (\mathbf{e}_{t_1} + \mathbf{e}_{t_2}). \tag{17}$$

This hint is optional and only contributes when the corresponding timestamps are available.

B.5 Bidirectional Entity Scoring

Using the contextualized slots $\hat{\mathbf{e}}_h, \hat{\mathbf{e}}_t, \hat{\mathbf{e}}_\tau$, the forward and backward TComplEx scores are:

$$\mathbf{s}^{\rightarrow(k)}_{\text{ent}} = \text{Score}_{\text{TComplEx}}(\hat{\mathbf{e}}_h, \hat{\mathbf{e}}_t, \mathbf{r}^{(k)}_{\text{ent}}, \hat{\mathbf{e}}_\tau), \tag{18}$$

$$\mathbf{s}^{\leftarrow(k)}_{\text{ent}} = \text{Score}_{\text{TComplEx}}(\hat{\mathbf{e}}_t, \hat{\mathbf{e}}_h, \mathbf{r}^{(k)}_{\text{ent}}, \hat{\mathbf{e}}_\tau). \tag{19}$$

They are fused by a learned scalar gate conditioned on the global summary:

$$\alpha_{\text{ent}}^{(k)} = \sigma(W_{\text{ent}}\mathbf{s}), \qquad \mathbf{s}_{\text{ent}}^{(k)} = \alpha_{\text{ent}}^{(k)}\mathbf{s}_{\text{ent}}^{\rightarrow(k)} + \big(1-\alpha_{\text{ent}}^{(k)}\big)\mathbf{s}_{\text{ent}}^{\leftarrow(k)}. \tag{20}$$

B.6 Bidirectional Time Scoring

Temporal candidates are scored analogously:

$$\mathbf{s}_{\text{time}}^{\rightarrow(k)} = \text{Score}_{\text{time}}(\hat{\mathbf{e}}_h, \hat{\mathbf{e}}_t, \mathbf{r}_{\text{time}}^{(k)}), \tag{21}$$

$$\mathbf{s}_{\text{time}}^{\leftarrow(k)} = \text{Score}_{\text{time}}(\hat{\mathbf{e}}_t, \hat{\mathbf{e}}_h, \mathbf{r}_{\text{time}}^{(k)}). \tag{22}$$

The two score vectors are fused by a gate conditioned on the global summary:

$$\alpha_{\text{time}}^{(k)} = \sigma(W_{\text{time}}\mathbf{s}), \qquad \mathbf{s}_{\text{time}}^{(k)} = \alpha_{\text{time}}^{(k)}\mathbf{s}_{\text{time}}^{\rightarrow(k)} + \big(1-\alpha_{\text{time}}^{(k)}\big)\mathbf{s}_{\text{time}}^{\leftarrow(k)}. \tag{23}$$

B.7 Working-Memory Update

After scoring, the model converts the entity and time scores into soft distributions:

$$\mathbf{p}_{\text{ent}}^{(k)} = \text{softmax}(\mathbf{s}_{\text{ent}}^{(k)}), \qquad \mathbf{p}_{\text{time}}^{(k)} = \text{softmax}(\mathbf{s}_{\text{time}}^{(k)}). \tag{24}$$

These are used to compute expected entity and time embeddings:

$$\bar{\mathbf{e}}_{\text{ent}}^{(k)} = \mathbf{p}_{\text{ent}}^{(k)}\mathbf{E}_{\text{ent}}, \qquad \bar{\mathbf{e}}_{\text{time}}^{(k)} = \mathbf{p}_{\text{time}}^{(k)}\mathbf{E}_{\text{time}}, \tag{25}$$

where $\mathbf{E}_{\text{ent}}$ and $\mathbf{E}_{\text{time}}$ are the entity and timestamp embedding tables from the TKBC model.

The two expected embeddings are summed, projected into a memory vector, and used to refine the latent state through an attention-based gated update:

$$\mathbf{m}^{(k)} = f_{\text{mem}}\big(\bar{\mathbf{e}}_{\text{ent}}^{(k)} + \bar{\mathbf{e}}_{\text{time}}^{(k)}\big), \tag{26}$$

followed by

$$\mathbf{u}^{(k)} = \text{Attn}\big(\mathbf{z}^{(k-1)}\uparrow, \mathbf{m}^{(k)}\uparrow, \mathbf{m}^{(k)}\uparrow\big)\downarrow, \tag{27}$$

and a gated residual fusion:

$$\gamma^{(k)} = \sigma\big(W_\gamma[\mathbf{z}^{(k-1)}; \mathbf{u}^{(k)}]\big), \tag{28}$$

$$\mathbf{z}^{(k)} = \gamma^{(k)} \odot \mathbf{z}^{(k-1)} + (1-\gamma^{(k)}) \odot \mathbf{u}^{(k)}. \tag{29}$$

This mechanism lets the model carry forward predictions from earlier hops and progressively refine its hypothesis.

B.8 Hop Aggregation and Training Objective

After K hops, a hop-selection distribution is computed from the global summary:

$$\boldsymbol{\beta} = \text{softmax}(W_{\text{hop}}\mathbf{s}). \tag{30}$$

The final predictions are weighted sums of the hop-specific scores:

$$\mathbf{s}_{\text{ent}} = \sum_{k=1}^{K} \beta_k \mathbf{s}_{\text{ent}}^{(k)}, \qquad \mathbf{s}_{\text{time}} = \sum_{k=1}^{K} \beta_k \mathbf{s}_{\text{time}}^{(k)}. \tag{31}$$

The final output is the concatenation $[\mathbf{s}_{\text{ent}}; \mathbf{s}_{\text{time}}]$, trained with cross-entropy against the gold answer distribution. The TKBC embeddings used for scoring can be kept fixed during QA training when the frozen setting is enabled.

B.9 Pseudocode

Algorithm 1 gives a compact PyTorch-style pseudocode matching the implementation.

Algorithm 1 SABET-QA forward pass.

Require: question tokens $\mathbf{x}$, mask $\mathbf{m}$, heads, tails, times, optional (t_1, t_2)
Ensure: concatenated entity/time scores

$\mathbf{H} \leftarrow \mathrm{LM}(\mathbf{x}, \mathbf{m})$
$\mathbf{T} \leftarrow$ text_proj$(\mathbf{H})$
$\mathbf{e}_h, \mathbf{e}_t, \mathbf{e}_\tau \leftarrow$ lookup(heads, tails, times)
$\hat{\mathbf{e}}_h, \hat{\mathbf{e}}_t, \hat{\mathbf{e}}_\tau \leftarrow$ slot_context$(\mathbf{T}, \mathbf{m}, \mathbf{e}_h, \mathbf{e}_t, \mathbf{e}_\tau)$
$\mathbf{s} \leftarrow$ summary$(\mathbf{T}, \mathbf{m})$
$\mathbf{z}^{(0)} \leftarrow$ rel_head$([\mathbf{s}; \hat{\mathbf{e}}_h; \hat{\mathbf{e}}_t; \hat{\mathbf{e}}_\tau])$
for $k = 1$ to K **do**
  if t_1, t_2 are available **then**
    $\mathbf{z}^{(k-1)} \leftarrow$ inject_temporal_hint$(\mathbf{z}^{(k-1)}, t_1, t_2)$
  end if
  $\mathbf{r}^{(k)} \leftarrow$ hop_proj$_k([\mathbf{z}^{(k-1)}; \mathbf{s}])$
  $\mathbf{r}^{(k)}_{\text{ent}} \leftarrow$ ent_proj$(\mathbf{r}^{(k)})$
  $\mathbf{r}^{(k)}_{\text{time}} \leftarrow$ time_proj$(\mathbf{r}^{(k)})$
  $\mathbf{s}^{(k)}_{\text{ent}} \leftarrow$ bidirectional_entity_score$(\hat{\mathbf{e}}_h, \hat{\mathbf{e}}_t, \hat{\mathbf{e}}_\tau, \mathbf{r}^{(k)}_{\text{ent}}, \mathbf{s})$
  $\mathbf{s}^{(k)}_{\text{time}} \leftarrow$ bidirectional_time_score$(\hat{\mathbf{e}}_h, \hat{\mathbf{e}}_t, \mathbf{r}^{(k)}_{\text{time}}, \mathbf{s})$
  if $k < K$ **then**
    $\mathbf{z}^{(k)} \leftarrow$ memory_update$(\mathbf{z}^{(k-1)}, \mathbf{s}^{(k)}_{\text{ent}}, \mathbf{s}^{(k)}_{\text{time}})$
  end if
end for
$\boldsymbol{\beta} \leftarrow \mathrm{Softmax}(W_{\text{hop}}\mathbf{s})$
$\mathbf{s}_{\text{ent}} \leftarrow \sum_{k=1}^{K} \beta_k \, \mathbf{s}^{(k)}_{\text{ent}}$
$\mathbf{s}_{\text{time}} \leftarrow \sum_{k=1}^{K} \beta_k \, \mathbf{s}^{(k)}_{\text{time}}$
return $[\mathbf{s}_{\text{ent}}; \mathbf{s}_{\text{time}}]$

C Dataset Statistics

Dataset	Train	Valid	Test	Total	Split Ratio (Tr/Va/Te)
CronQuestions	350,000	30,000	30,522	410,522	85.3% / 7.3% / 7.4%
Complex-CronQuestions	35,795	5,020	5,528	46,343	77.2% / 10.8% / 11.9%
MultiTQ	386,787	57,979	54,584	499,350	77.5% / 11.6% / 10.9%
TimeQuestions	6,970	3,236	3,237	13,443	51.8% / 24.1% / 24.1%

Table 6: Statistics of the temporal question answering datasets used in our experiments.

Dataset	Entities	Relations	Timestamps	Train Facts	Valid Facts	Test Facts	Total Facts
CronQuestions	125,726	406	9621	323,635	5000	5000	333,635
Complex-CronQuestions	125,726	406	9621	323,635	5000	5000	333,635
MultiTQ	10,488	502	4,017	322,958	69,224	69,147	461,329
TimeQuestions	118,010	884	1,636	227,564	4,997	4,998	240,597

Table 7: Statistics of the temporal knowledge graphs associated with each benchmark.

We evaluate SABET-QA on four benchmark datasets for temporal question answering: CronQuestions (Saxena et al., 2021), Complex-CronQuestions (Chen et al., 2022), MultiTQ (Chen et al., 2023), and TimeQuestions (Jia et al., 2021).

CronQuestions and Complex-CronQuestions are synthetic benchmarks derived from temporal Wikidata facts and focus on compositional temporal reasoning. MultiTQ contains large-scale automatically generated temporal questions involving diverse temporal operators and reasoning patterns. TimeQuestions consists of natural-language temporal questions collected from real-world sources and provides a complementary evaluation setting with realistic temporal information needs.

CronQuestions			
Answer Type	Train	Valid	Test
Entity	225,672	19,362	19,524
Time	124,328	10,638	10,476
Total	350,000	30,000	30,000

(a) Answer types in CronQuestions.

Complex-CronQuestions			
Answer Type	Train	Valid	Test
Entity	23,029	3,340	3,382
Time	12,766	1,680	1,624
Total	35,795	5,020	5,006

(b) Answer types in Complex-CronQuestions.

MultiTQ			
Answer Type	Train	Valid	Test
Entity	267,155	40,565	38,700
Time	119,632	17,414	15,884
Total	386,787	57,979	54,584

(c) Answer types in MultiTQ.

TimeQuestions			
Answer Type	Train	Valid	Test
Entity	4,589	2,292	2,340
Time	2,381	944	897
Total	6,970	3,236	3,237

(d) Answer types in TimeQuestions.

Table 8: Answer type distributions across the benchmarks.

Setting	**Value**
Optimizer	Adam (Kingma and Ba, 2017)
Initial learning rate	2×10^{-4} (6×10^{-4} for Timequestions Dataset)
Maximum epochs	20 (50 for Timequestions Dataset)
Training batch size	150
Validation batch size	150
Validation frequency	Every epoch
Learning-rate schedule	Linear warm-up + cosine annealing
Warm-up steps	$\min(200,\ 0.1 \times$ total steps$)$
Warm-up start factor	0.1
Cosine minimum LR	$0.01\times$ initial LR
Evaluation metrics	Hits@1, Hits@10
Checkpoint selection	Best validation Hits@1
Pretrained KG embeddings	Loaded from dataset-specific checkpoint
KG embedding update	Frozen during QA training (Except for Ablation Studies)
Language model update	Frozen during QA training (Except for Ablation Studies)

Table 9: Training settings used for all experiments unless otherwise noted.

Table 6 summarizes the statistics of the question-answering datasets. Table 7 reports the characteristics of the associated temporal knowledge graphs. The datasets vary considerably in scale, ranging from 13K questions in TimeQuestions to nearly 500K questions in MultiTQ. Likewise, the underlying temporal knowledge graphs differ substantially in their numbers of entities, relations, timestamps, and temporal facts.

Table 8 further reports the distribution of answer types across the benchmarks. All datasets contain both entity-answer and timestamp-answer questions, providing a comprehensive evaluation of temporal reasoning capabilities across heterogeneous settings.

D Training Settings

Table 9 summarizes the training configuration used in our experiments. All models were implemented in PyTorch (Paszke et al., 2019). The language model and pretrained KG embeddings were frozen during QA training.